Description | Testcase | Test Result | Accepted ×

2891. Method Chaining

Solved

Easy | Companies

DataFrame `animals`

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| name        | object  |
| species     | object  |
| age         | int     |
| weight      | int     |
+-------------+---------+
```

Write a solution to list the names of animals that weigh **strictly more than** `100` kilograms.

Return the animals sorted by weight in **descending order**.

The result format is in the following example.

101 | 14 | 2 Online

Code

Pandas | Auto

```python
import pandas as pd

def findHeavyAnimals(animals: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(animals)
    df = df[df["weight"]>100][["name","weight"]].sort_values(by="weight",
ascending = False)
    df = df[["name"]]
    return df
```

Saved | Ln 1, Col 1

Description | Testcase | Test Result | Accepted ×

2890. Reshape Data: Melt

Solved

Easy | Companies | Hint

DataFrame `report`

```
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| product     | object |
| quarter_1   | int    |
| quarter_2   | int    |
| quarter_3   | int    |
| quarter_4   | int    |
+-------------+--------+
```

Write a solution to **reshape** the data so that each row represents sales data for a product in a specific quarter.

The result format is in the following example.

109 | 8 | 1 Online

Code

Pandas | Auto

```python
import pandas as pd

def meltTable(report: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(report)
    df_melt = df.melt(id_vars=["product"],var_name="quarter",value_name="sales")
    return df_melt

```

Saved | Ln 5, Col 56

Description | Testcase | Test Result | Accepted

2889. Reshape Data: Pivot

Solved

Easy | Companies | Hint

DataFrame `weather`

```
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| city        | object |
| month       | object |
| temperature | int    |
+-------------+--------+
```

Write a solution to **pivot** the data so that each row represents temperatures for a specific month, and each city is a separate column.

The result format is in the following example.

Example 1:

133 | 22 | 2 Online

Code

Pandas | Auto

```python
import pandas as pd

def pivotTable(weather: pd.DataFrame) -> pd.DataFrame:
    return weather.pivot(index="month",columns = "city",values="temperature")

```

Saved | Ln 4, Col 63

Description | Testcase | Test Result | Accepted

2888. Reshape Data: Concatenate

Solved

Easy | Companies | Hint

```
DataFrame df1
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| student_id  | int    |
| name        | object |
| age         | int    |
+-------------+--------+

DataFrame df2
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| student_id  | int    |
| name        | object |
| age         | int    |
+-------------+--------+
```

86 | 16 | 3 Online

Code

Pandas | Auto

```python
import pandas as pd

def concatenateTables(df1: pd.DataFrame, df2: pd.DataFrame) -> pd.DataFrame:
    df = pd.concat([df1,df2],ignore_index=True)
    return df

```

Saved | Ln 4, Col 43

Description | Testcase | Test Result | Accepted

2887. Fill Missing Data

Solved

Easy | Companies | Hint

DataFrame `products`

```
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| name        | object |
| quantity    | int    |
| price       | int    |
+-------------+--------+
```

Write a solution to fill in the missing value as `0` in the `quantity` column.

The result format is in the following example.

Example 1:

Input:

86 | 25 | 3 Online

Code

Pandas | Auto

```python
import pandas as pd

def fillMissingValues(products: pd.DataFrame) -> pd.DataFrame:
    products["quantity"] = products["quantity"].fillna(0)
    return products

```

Saved | Ln 4, Col 11

Description | Testcase | Test Result | Accepted

2886. Change Data Type

Solved

Easy | Companies | Hint

DataFrame `students`

```
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| student_id  | int    |
| name        | object |
| age         | int    |
| grade       | float  |
+-------------+--------+
```

Write a solution to correct the errors:

The `grade` column is stored as floats, convert it to integers.

The result format is in the following example.

84 | 18 | 3 Online

Code

Pandas | Auto

```python
import pandas as pd

def changeDatatype(students: pd.DataFrame) -> pd.DataFrame:
    students["grade"] = students["grade"].astype(int)
    return students

```

Saved | Ln 4, Col 40

Description | Testcase | Test Result | Accepted

2885. Rename Columns

Solved

Easy | Companies | Hint

```
DataFrame students
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| id          | int    |
| first       | object |
| last        | object |
| age         | int    |
+-------------+--------+
```

Write a solution to rename the columns as follows:

- `id` to `student_id`
- `first` to `first_name`
- `last` to `last_name`

79 | 21 | 2 Online

Code

Pandas | Auto

```python
import pandas as pd

def renameColumns(students: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(students)
    df = df.rename(columns={"id":"student_id","first":"first_name",
"last":"last_name","age":"age_in_years"})
    return df
```

Saved | Ln 5, Col 108

Description | Testcase | Test Result | Accepted

```
| Column Name | Type   |
+-------------+--------+
| name        | object |
| salary      | int    |
+-------------+--------+
```

A company intends to give its employees a pay rise.

Write a solution to **modify** the `salary` column by multiplying each salary by 2.

The result format is in the following example.

Example 1:

Input:

```
DataFrame employees
+---------+--------+
| name    | salary |
+---------+--------+
| Jack    | 19666  |
| Piper   | 74754  |
| Mia     | 62509  |
```

90 | 16 | 2 Online

Code

Pandas | Auto

```python
import pandas as pd

def modifySalaryColumn(employees: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(employees)
    df["salary"] = df["salary"]*2
    return df
```

Saved | Ln 4, Col 19

Description | Testcase | Test Result | Accepted

2883. Drop Missing Data

Solved

Easy | Companies | Hint

```
DataFrame students
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| student_id  | int    |
| name        | object |
| age         | int    |
+-------------+--------+
```

There are some rows having missing values in the `name` column.

Write a solution to remove the rows with missing values.

The result format is in the following example.

Code | Code Sample

Pandas | Auto

```
import pandas as pd

def dropMissingData(students: pd.DataFrame) -> pd.DataFrame:
    return students.dropna(subset = ["name"])
```

Description | Testcase | Test Result | Accepted

2882. Drop Duplicate Rows

Solved

Easy | Companies | Hint

```
DataFrame customers
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| customer_id | int    |
| name        | object |
| email       | object |
+-------------+--------+
```

There are some duplicate rows in the DataFrame based on the `email` column.

Write a solution to remove these duplicate rows and keep only the **first** occurrence.

The result format is in the following example.

Code

Pandas | Auto

```python
import pandas as pd

def dropDuplicateEmails(customers: pd.DataFrame) -> pd.DataFrame:
    return customers.drop_duplicates(subset=["email"])
```

Description | Testcase | Test Result | Accepted ×

2881. Create a New Column

Solved

Easy | Companies | Hint

```
DataFrame employees
+-------------+--------+
| Column Name | Type.  |
+-------------+--------+
| name        | object |
| salary      | int.   |
+-------------+--------+
```

A company plans to provide its employees with a bonus.

Write a solution to create a new column name `bonus` that contains the **doubled values** of the `salary` column.

The result format is in the following example.

102 | 24 | 5 Online

Code

Pandas | Auto

```python
import pandas as pd

def createBonusColumn(employees: pd.DataFrame) -> pd.DataFrame:
    employees["bonus"] = employees["salary"] * 2
    return employees


```

Saved | Ln 5, Col 21

Description | Testcase | Test Result | Accepted ×

2880. Select Data

Solved

Easy | Companies | Hint

```
DataFrame students
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| student_id  | int    |
| name        | object |
| age         | int    |
+-------------+--------+
```

Write a solution to select the name and age of the student with `student_id = 101`.

The result format is in the following example.

Code

Pandas | Auto

```python
import pandas as pd

def selectData(students: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(students)
    return df.loc[df["student_id"]==101,["name","age"]]

```

Description | Testcase | Test Result | Accepted ×

2879. Display the First Three Rows

Solved

Easy | Companies | Hint

```
DataFrame: employees
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| employee_id | int    |
| name        | object |
| department  | object |
| salary      | int    |
+-------------+--------+
```

Write a solution to display the **first** 3 rows of this DataFrame.

Example 1:

Input:

Code

Pandas | Auto

```python
import pandas as pd

def selectFirstRows(employees: pd.DataFrame) -> pd.DataFrame:
    df = pd.DataFrame(employees)
    return df.head(3)
```

Description | Testcase | Test Result | Accepted

2878. Get the Size of a DataFrame

Solved

Easy · Companies · Hint

```
DataFrame players:
+-------------+--------+
| Column Name | Type   |
+-------------+--------+
| player_id   | int    |
| name        | object |
| age         | int    |
| position    | object |
| ...         | ...    |
+-------------+--------+
```

Write a solution to calculate and display the **number of rows and columns** of `players`.

Return the result as an array:

`[number of rows, number of columns]`

146 · 31 · 1 Online

Code

Pandas · Auto

```python
import pandas as pd

def getDataframeSize(players: pd.DataFrame) -> List[int]:
    df = pd.DataFrame(players)
    row,cols = df.shape
    return[row,cols]
```

Saved · Ln 6, Col 15

Description | Testcase | Test Result

Input:

```
student_data:
[
  [1, 15],
  [2, 11],
  [3, 11],
  [4, 20]
]
```

Output:

```
+------------+-----+
| student_id | age |
+------------+-----+
| 1          | 15  |
| 2          | 11  |
| 3          | 11  |
| 4          | 20  |
+------------+-----+
```

Explanation:

A DataFrame was created on top of student_data, with two columns named `student_id` and `age`.

277 | 48 | 13 Online

Code

Pandas | Auto

```python
import pandas as pd

def createDataframe(student_data: List[List[int]]) -> pd.DataFrame:
    student_data = pd.DataFrame(student_data,columns = ["student_id","age"])
    return student_data

```

Saved

Ln 5, Col 24